

Binaries and Memory — A Readable Companion to the OTP Docs

Chapter 3 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why memory layout matters in Erlang

In Erlang, the way you represent data decides both its size and whether a message copies it. The same 1 KB of text can cost 1 KB or 16 KB, and it can be shared between processes or copied into each one, depending only on the type you picked.

Sizes are measured in words: 8 bytes on a 64-bit system (check with `erlang:system_info(wordsize)`).

Term	Size
Small integer, atom, local pid	1 word
Float (64-bit)	3 words
Tuple	2 words + elements
List	1 word + 1 per element + elements
String as a list of characters	1 word + 2 per character
Small map (≤ 32 keys)	5 words + keys and values
Binary	3–6 words + the bytes

So `"hello world"` as a character list costs 23 words (184 bytes). As a binary it is 11 bytes plus a small header. Elixir strings are already UTF-8 binaries; charlists (`~c"..."`) are the list form and cost the same as Erlang strings.

2. The building blocks

Four kinds of binary

Internally the runtime has four binary representations. You never choose between them directly, but knowing which one you have explains most binary memory surprises.

Kind	Where the bytes live	When you get it
Heap binary	On the process heap	Up to 64 bytes; copied on send like any term
Refc binary	Outside all heaps, reference-counted	Over 64 bytes; shared on send, only a small handle is copied
Sub-binary	Points into another binary	Slicing or matching out part of a binary
Match context	Internal pointer used during matching	Binary pattern matching in loops

The docs note that binary handling was rewritten in OTP 27 and the Efficiency Guide text has not fully caught up. The internal names may shift, but the behaviour this chapter relies on is the same.

iodata: text you never have to concatenate

`iodata()` is a binary, or a list that nests bytes, binaries and more `iodata`. Files, sockets and ports accept it directly, so you can build output as a tree and skip concatenation entirely:

```
Html = [<<"<li>">>, Name, <<"</li>">>],
ok = gen_tcp:send(Sock, [Header, Html, Footer]).
```

Call `iolist_to_binary/1` only when you really need one flat binary, such as a hash input or a map key.

Atoms are forever

The atom table is not garbage-collected. By default it holds 1,048,576 atoms (change with `+t`), and each atom can be at most 255 characters. Atoms are ideal for a fixed vocabulary of tags and names, and dangerous for anything derived from input.

3. How the runtime actually does it

Refc binaries and their handles

A large binary is two pieces: the bytes, stored off-heap with a reference count, and a small handle (a `ProcBin`) on each process heap that uses it. The count drops only when a handle is garbage-collected. So a large binary is freed only after every process that ever touched it has run a garbage collection since dropping it.

Sub-binaries keep the whole parent alive

Slicing a large binary does not copy. The slice points into the original, so a 16-byte slice of a 10 MB binary keeps all 10 MB alive for as long as the slice exists. `binary:referenced_byte_size/1` shows how much a binary is really holding onto.

The append optimization

Building a binary with `<<Acc/binary, More/binary>>` in a loop is efficient: the runtime over-allocates and appends in place. That only works while there is a single handle with a single reference to the accumulator. The following force a copy on the next append:

- keeping and using an older version of the accumulator
- sending the accumulator in a message
- inserting it into ETS
- passing it to a port
- pattern matching on it

The match optimization

Recursive matching like `<<H, T/binary>>` is compiled to reuse one match context instead of creating a sub-binary per step. Compile with `erlc +bin_opt_info` to see where the optimization applied and where it didn't.

4. Plausible but wrong

4.1 Prepending to a binary accumulator

```
encode([], Acc) -> Acc;
encode([X | Xs], Acc) -> encode(Xs, <<(esc(X))/binary, Acc/binary>>).
```

Prepending cannot use the append optimization, so every step copies the whole accumulator: $O(n^2)$ for n items. It looks exactly like the efficient version. Fix: append (`<<Acc/binary, (esc(X))/binary>>`) over reversed input, or collect an iolist and flatten once at the end.

4.2 Quadratic string building with ++

```
render(Rows) ->
  lists:foldl(fun(R, Acc) -> Acc ++ format_row(R) end, "", Rows).
```

`++` copies its whole left operand, so this is $O(n^2)$, and the result is a character list at 16 bytes per character. Fix: `[format_row(R) || R <- Rows]` is already valid iodata; hand it straight to the socket or file.

4.3 Keeping a tiny slice of a huge binary

```
handle_info({http_body, Body}, State) ->  %% Body is 10 MB
  <<_:32/binary, SessionId:16/binary, _/binary>> = Body,
  {noreply, State#{last_session => SessionId}}.
```

`SessionId` is a sub-binary, so the process state now pins the full 10 MB for as long as it keeps it. Repeat this per request and memory climbs with no obvious cause. Fix: `binary:copy(SessionId)` before storing it.

The docs caution that copying only helps when nothing else still references the large binary, so confirm with `binary:referenced_byte_size/1` first.

4.4 The router that never collects

```
% long-lived process forwarding large payloads
loop(Routes) ->
  receive
    {msg, Key, Payload} ->
      maps:get(Key, Routes) ! Payload,
      loop(Routes)
  end.
```

Each payload is a large refc binary, but the router allocates almost nothing on its own heap, so it rarely garbage-collects. The handles pile up and every payload it has ever forwarded stays alive. Binary memory grows while the process itself looks small. Fix: hibernate the process when it goes idle, lower its `fullsweep_after` through `spawn_opt`, or send payloads directly so the router never touches them.

4.5 Atoms from input

```
handle_request(#{<<"action">> := Action} = Req) ->
  dispatch(binary_to_atom(Action), Req).
```

Every distinct value a client sends becomes a permanent atom. An attacker, or just a long-running system, eventually reaches 1,048,576 atoms and the node goes down. Fix: `binary_to_existing_atom/1` (fails unless the atom already exists), or match the binary directly: `dispatch(<<"create">>, Req) -> ...`

4.6 Breaking the append optimization by accident

```
build([], Acc) -> Acc;
build([X | Xs], Acc) ->
  Acc1 = <<Acc/binary, X:32>>,
  log_size(byte_size(Acc)),      % still uses the old Acc
  build(Xs, Acc1).
```

Using `Acc` after creating `Acc1` means two live versions of the accumulator, so the next append must copy. Fix: use only the newest version (`byte_size(Acc1)`), and check with `+bin_opt_info`.

5. Review checklist and sources

- ☐ Text in hot paths is binaries or iodata, not character lists
- ☐ Output is built as iodata and written once; no `++` or binary concatenation in loops
- ☐ Binary accumulators only ever append, and only the newest version is used
- ☐ Small slices of large binaries that are stored long-term are copied (after checking `referenced_byte_size`)
- ☐ Long-lived processes that touch large binaries hibernate or garbage-collect regularly

- ❑ No atoms are created from external input
- ❑ `+bin_opt_info` has been checked on binary-heavy modules

Sources

- Constructing and Matching Binaries, Efficiency Guide: <https://www.erlang.org/doc/system/binaryhandling.html>
- Memory Usage, Efficiency Guide: <https://www.erlang.org/doc/system/memory.html>
- System Limits: https://www.erlang.org/doc/system/system_limits.html
- binary module reference: <https://www.erlang.org/doc/apps/stdlib/binary.html>
- erlang module reference: <https://www.erlang.org/doc/apps/erts/erlang.html>